

测试需求分析与测试计划

美国项目管理研究所(Project Management Institute, PMI)认为,项目管理是在项目活动中运用一系列的知识、技能、工具和技术,以满足并超过相关利益者对项目的要求和期望。它实际指出了项目管理涉及的范畴和需要达到的目标。使项目顺利进行并达到预期的效果,这就是项目管理所应该实现的基本目标。那么,软件项目管理的目标就是为了使软件项目能够按照预定的成本、进度、质量顺利完成,而对成本、资源、进度、质量、风险等进行分析和控制的活动。

对于软件测试项目管理,在概念上和一般项目管理没有区别,但所管理的具体内容、所采用的具体技术和工具是不同的,而且关注点也不一样。软件测试项目的管理,始终围绕如何保证产品质量开展工作,防范软件开发中所存在的各种质量风险,密切跟踪和分析缺陷,最终在合理的成本和进度控制下能够有效地完成所有的测试工作,帮助团队发布满足用户要求和期望的、可维护的、高质量的软件产品。

在软件测试项目管理中,主要的工作就是做好计划和监控:测试计划的制定和执行计划的监督与控制。测试计划涉及面比较广,主要包括以下内容。

- (1) 测试目标和测试范围的确定,包括具体测试项及其优先级;
- (2) 识别测试风险,并采取相对应的测试策略,包括测试方法和工具的选择;
- (3) 测试工作量估算、测试资源分配;
- (4) 测试阶段划分、里程碑定义和进度表编制;
- (5) 最终交付内容。

但要做好计划,需要做好测试需求分析,即明确测试的范围和测试项,这也是测试工作量估算的基础,更是测试资源计划、进度计划的基础。

10.1 测试的目标和准则

测试项目计划的整体目标是为了确定测试的任务、所需的各种资源和投入、预见可能出现的问题和风险,以指导测试的执行,最终实现测试的目标,保证软件产品的质量。在项目的管理过程中,经常碰到的问题

是：等待做的任务比较多，但人力资源和时间受到限制，要完成所有的任务几乎是不可能的。这时候要解决的就是为各项任务建立优先级，这样就可以根据优先级高低，先后处理各项任务，降低测试的风险，以最小的代价获得尽可能高的质量。

1. 确定测试目标

(1) 为测试各项活动制定一个现实可行的、综合的计划，包括每项测试活动的对象、范围、方法、进度和预期结果；

(2) 定义测试项目中每个角色的责任和工作内容；

(3) 开发有效的测试模型，选择合适的测试方法，能正确地验证正在开发的软件系统；

(4) 确定测试所需要的时间和资源，以保证其可获得性、有效性；

(5) 确立每个测试阶段测试完成以及测试成功的标准、要实现的目标；

(6) 识别出测试活动中各种风险，并消除可能存在的风险，降低那些不可能消除的风险所带来的损失；

(7) 基于风险评估和资源、时间的限制，制定有效的测试策略，在可预见的、可接受的风险前提下，保证项目测试任务按时完成。

2. 软件测试项目的出入准则

为了保证测试实施能按计划执行，必须确认测试在满足什么外部条件下才能开始，这就要求在测试计划中定义软件测试项目及其各个阶段的进入准则，然后再定义测试项目及其各个阶段的结束准则。例如，测试项目的结束准则包括测试用例评审准则、测试执行结束准则、Bug 描述和处理准则、文档模板和质量评估准则等，而测试项目的进入准则有以下几点。

(1) 清楚了解项目的整体计划框架，才能够制定测试计划。

(2) 完成需求规格说明书评审：只有了解到用户具体的、实际的需求，才能分析和确定测试需求和测试范围。

(3) 技术知识或业务知识的储备，无论是业务领域发生变化还是新技术的引入，测试人员都需要事先做好知识准备，包括对测试人员进行必要的培训。

(4) 标准环境：建立用户使用环境或业务运行环境一致或非常接近的测试环境。

(5) 技术设计文档是测试用例设计的重要参考资料，帮助测试人员了解系统的技术实现以及系统可能存在的薄弱环节等。

(6) 足够的资源，包括人力资源、硬件资源、软件资源和其他环境资源。

(7) 人员组织结构，项目经理、测试组长、成员等责任及其之间的关系已确定。

3. 测试项目管理的原则

要管理好软件测试项目，首先要制定好测试管理流程和测试规范，明确定义测试过程中各种活动、技术标准、度量指标和相应的文档模板。其次，要有正确的管理方法，包括对风险、进度和质量的管理。最后，在规范的测试流程和客观的评价标准基础之上，就要从软件测试项目的人员角色和责任、畅通的交流渠道、完善的奖惩体系等各方面着手，提高测试团队的作战能力。

(1) 可靠的需求。测试的需求是经各方一致同意的、可实现的并在文档中清楚地、完整地、和详细地描述。

(2) 能够适应开发过程模型。例如，当采用快速开发模型或敏捷方法时，为了能够应对需求的变化，面对频繁的软件发布，测试人员需要和开发人员同步工作，并尽力实现自动化测试。

(3) 充分测试和尽早开始测试。每次改错或变更后,不仅要测试修改的地方,而且应该进行足够的回归测试。

(4) 合理的时间表。为测试设计、执行、变更后再测试以及测试结果分析等留出足够的时间,进行周密计划,不应使用突击的办法来完成项目。

(5) 充分沟通。不仅在测试团队内部做好沟通,而且要与开发人员、产品经理、市场人员甚至客户等进行有效沟通,并采用合适的通信手段,如电话、即时消息(IM)、远程在线会议系统和电子邮件等。

(6) 基于数据库的测试管理系统,通过这个系统有效地管理测试计划、测试用例、测试任务、缺陷和测试报告等,确保及时的管理和良好的协作。

10.2 测试需求分析

测试人员掌控了软件项目的背景、产品测试目标和准则,阅读相关软件需求文档,参与需求评审,以及掌握了第2章所学的产品质量特性知识,在这些基础之上,可以进行测试的需求分析,即包括下面这些工作。

- (1) 明确测试范围,了解哪些功能点要测试、哪些功能点不需要测试;
- (2) 知道哪些测试目标优先级高、哪些目标优先级低;
- (3) 要完成哪些相应的测试任务才能确保目标的实现。

然后才能估算测试的工作量,安排测试的资源 and 进度。测试需求分析是测试设计和开发测试用例的基础,测试需求分析得越细,对测试用例的设计质量的帮助越大,详细的测试需求还是衡量测试覆盖率的主要依据。只有在做好测试需求的基础上,才能规划项目所需的资源、时间以及所存在的风险等。

10.2.1 测试需求分析的基本方法

无论是功能测试,还是非功能性测试,其测试需求的分析都有如下两个基本的出发点。

(1) 从客户角度进行分析:通过业务流程、业务数据、业务操作等分析,明确要验证的功能、数据、场景等内容,从而确定业务方面的测试需求。

(2) 从技术角度分析:通过研究系统架构设计、数据库设计、代码实现等,分析其技术特点,了解设计和实现要求,包括系统稳定可靠、分层处理、接口集成、数据结构、性能等方面的测试需求。

如果有完善的需求文档(如产品功能规格说明书),那么功能测试需求可以根据需求文档,再结合前面分析和自己的业务知识等,比较容易确定功能测试的需求。如果缺乏完善的需求文档,就需要借助启发式分析方法,从系统业务目标、结构、功能、数据、运行平台、操作等多方面进行综合分析,了解测试需求,并通过和用户、业务人员、产品经理或产品设计人员、开发人员等沟通,逐步让测试需求清晰起来。

(1) 业务目标:所有要做的功能特性都不能违背该系统要达到的业务目标,多问问如何更好地达到这些业务目标,如何验证是否实现这些业务目标?

(2) 系统结构:产品是如何构成的?系统有哪些组件、模块?模块之间有什么样的关系?有哪些接口?各个组件又包含哪些信息?

(3) 系统功能:产品能做什么事、处理哪些业务?处理某些业务时由哪些功能来支撑、形

成怎样的处理过程? 处理哪些错误类型? 有哪些 UI 来呈现这些功能?

(4) 系统数据: 产品处理哪些数据? 最终输出哪些用户想要的结果? 哪些数据是正常的? 又有哪些异常的数据? 输入数据是如何被转化、传递的? 这中间有哪些过渡性数据? 输出数据格式有什么要求? 输出数据存储在哪里?

(5) 系统运行的平台: 系统运行在什么硬件上? 什么操作系统? 有什么特殊的环境配置? 是否依赖第三方组件?

(6) 系统操作: 有哪些操作角色? 什么场景下使用? 不同角色、场景有什么不同? 有哪些是交集的?

上面这些分析,更多是从测试对象本身来进行分析,还包括用户角色分析、用户行为分析、用户场景分析等。还可以通过其他方面的资料,帮助我们更好地完成测试的需求分析。

(1) 对竞争产品进行对比分析,明确测试的重点。

(2) 质量存在哪些风险,包括安全性漏洞等。

(3) 对过去类似产品或本产品上个版本所发现的缺陷进行分析,总结缺陷出现的规律,看看有没有漏掉的测试需求。

(4) 在易用性、用户体验上有什么特别的需求需要验证?

(5) 管理者或市场部门有没有事先特定的声明?

(6) 有没有相应的行业规范、特许质量标准?

测试需求分析过程,可以从质量要求出发,来展开测试需求分析,如从功能、性能、安全性、兼容性等各个质量要求出发,不断细化其内容,挖掘其对应的测试需求,覆盖质量要求。也可以从开发需求(如产品功能特性点、敏捷开发的用户故事)出发,针对每一条开发需求形成已分解的测试项,结合质量要求,这些测试项再扩展为测试任务,这些测试任务包括具体的功能性测试任务和非功能性测试任务。在整理测试需求时,需要分类、细化、合并并按照优先级进行排序,形成测试需求列表。

10.2.2 测试需求分析的技术

在软件测试需求分析过程中,可以采用有效的问题分析技术来帮助提高测试需求的有效性和工作效率。从测试需求分析来看,应力求通过与各相关干系人的沟通,收集足够的、有价值的信息或数据,借助下列途径来达到良好的分析效果,如:

(1) 通过提炼,抓住主要线索,或作为整体来进行分析,使测试需求分析简单化;

(2) 通过业务需求或功能层次的整理,使测试需求分析结构化、层次化;

(3) 通过绘制业务流程图、数据流程图等,使测试需求分析可视化;

(4) 通过类比、隐喻,加强用户需求的理解,更好地转化为测试需求。

在测试需求的分析中,能采用静态分析技术与动态分析技术、定性分析技术和定量分析技术,其中以静态分析技术、定性分析技术为主,但产品性能、用户行为分析和用户体验分析等也常采用定量分析技术。有时,会采用综合分析技术、模型分析技术等。

在测试需求分析时,产品本身往往处于需求分析和设计过程中,静态分析技术是常用的分析技术。静态分析技术包括:

(1) 通过系统建模语言(SysML)的需求图,可以更好地分析各项需求之间的关系,比较容易确定测试需求的边界。

(2) 通过状态图、活动图更容易列出测试场景,了解状态转换的路径和条件,哪些是重要

测试场景等。

(3) 实体关系图：可以明确测试的具体对象(实体)及其之间的关系,进行相关分析。

(4) 鱼骨图法、思维导图等,有一个清晰的分析思维过程,迅速展开测试需求,随时补充测试需求等。

(5) 代码复杂度静态分析工具,代码越复杂,测试的投入也需要越多。

(6) 还可以用一些普通工具,如检查表。

(7) 脑力激荡法,让大家发散思维,相互启发,不会错过任何测试需求。

而动态分析技术应用相对少一些,但在一些应用场景的分析中还是有帮助的,如前面提到的竞争对手产品分析,这是一种动态分析技术,通过操作竞争对手产品,全面了解相同业务的需求,在功能、逻辑、界面等各个方面深入挖掘测试需求。同样道理,需求原型分析技术——基于开发已构建的原型来进行测试需求分析,更能直观地理解产品,进而有助于测试需求的分析,达到类似效果。可以采用仿真技术、模拟技术、角色扮演等手段,也能帮助测试需求的分析。

10.2.3 功能测试范围分析

在分析测试范围时,一般先进行功能测试的范围分析,然后再进行非功能性测试的范围分析。对于功能测试,可以借助业务流程图、功能框图等来帮助进行测试的需求分析。在面向对象的软件开发中,也可借助 UML 用例图、活动图、协作图和状态图来进行功能测试范围分析。例如,针对 Web 应用系统,可以列出一些共性的测试需求。

(1) 用户登录,登录的用户名、口令能否保存? 口令忘了,能否找回来? 允许登录失败的次数是否有限制? 口令字符有没有严格要求(如长度、大小写、特殊字符)? 是否硬性规定经过一段时间后必须改变口令?

(2) 站点地图和导航条。每个网站都需要站点地图,让用户一看就能了解网络内容,而且当新用户在网站中迷失方向时,站点地图可以引导用户进行浏览,找到所想访问的内容。需要验证站点地图每一个链接是否存在而且正确,有没有涵盖站点上所有内容的链接。是否每个页面都有导航条? 导航条是否一致、直观?

(3) 链接跳转正确,即链接地址正确,并能显示正常、自然,不要给人突然的感觉。

(4) 表单,各项输入是必需的、合理的,各项操作正常,对于错误的输入有准确、适当的提示,并完成最后的提交。提交后,返回提交内容的显示,使用户放心。如用户通过表单进行注册,能输入用户名、口令、地址、电话、爱好等各种信息,当格式、内容不对或不符时,及时给予提示,在用户提交信息后,进一步检查各项内容的正确性,然后写入数据库、返回注册成功的消息。

(5) 数据校验,根据业务规则和流程对用户输入数据进行校验,是许多系统不可缺少的。通过列表选择、规则提示或在线帮助,能很好地解决问题。

(6) Cookie,在 Web 应用中到处可见,用来保存用户注册、访问和其他本地客户端信息,所保存的信息要加密,并能及时更新。Cookie 被删除后,可以被重建。

(7) Session,是否安全、稳定,而且占用较少的资源。

(8) SSL、防火墙等的测试。使用了 SSL,浏览器地址栏中 URL 的“http:”就变为“https:”,服务器的连接端口号则由 80 变为 433,应用程序接口 API 也要和页面保持一致。防火墙支持更多的设置,包括代理、验证方式、超时等。

(9) 接口测试,与数据库服务器、第三方产品接口(如电子商务网站信用卡验证)的测试,包括接口错误代号和列表。

10.2.4 非功能性的系统测试需求

对于非功能性的系统测试,主要目的是验证软件系统的整体性能等是否满足其产品设计规格所指定的要求,涉及非功能性的质量需求,包括系统性能、安全性、兼容性、扩充性等的测试,可能还会涉及第三方产品的集成测试。对于每一个应用软件系统,非功能特性的质量需求都是存在的,这类测试需求会因不同的项目类型差异比较大,这些需求的程度、重要性不同,这也要求为非功能性测试需求设置优先级,下面就做一个简单的分析。

(1) 纯客户端软件,如字处理软件、下载软件、媒体(音频/视频)播放软件等在系统非功能性测试要求上是最低的,对性能、容错性、稳定性等有一定的要求,如占用较少的系统资源(CPU 和内存),而且能运行在不同的操作系统上,一般分为 Windows 版、Linux 版和 Mac 版等。在 Windows 上要支持 Windows XP、Windows 7 和 Windows 8 等。

(2) 纯 Web(B/S)应用系统,如门户网站、个人博客网站、网络信息服务等在系统非功能性测试上要求较高,特别强调性能和可用性,对安全性有一定的要求,主要是保证数据的备份和登录权限。性能要求好,可以允许大量并发用户的访问,而且用户在任何时刻可以访问,即每周 7 天,每天 24 小时(7×24)运行。

(3) 客户/服务器(C/S)应用系统,如邮件系统、群件或工作流系统、即时消息系统等在系统非功能性测试需求上与 Web 应用系统接近,也可能出现大量并发访问的用户,但安全性相对好些,客户端是特定的开发软件,相对于浏览器来说,对端口、协议等的限制比较容易做到。

(4) 大型复杂企业级系统涉及面广、集成性强,包括 B/S、C/S、数据库、目录服务、服务器集群、XML 接口等各个子系统。在系统非功能性测试需求上,这类系统要求最高,不论是在性能、可用性方面,还是在安全性、可靠性等方面,都有很高的要求。

系统非功能性测试的需求在不同应用领域也体现出较大差异。如网上银行、信用卡服务等系统,其安全性、可用性和可靠性等多方面的测试至关重要,因为这方面的缺陷很可能会给用户造成较大的损失。这些系统需要得到充分的安全性测试、容错性测试和负载测试。多数情况下,还需要独立第三方的安全认证。

而对于局域网内的企业级应用来说,有关权限控制、口令设置等安全性测试依然重要,但兼容性测试就相对简单,因为可以指定某些特定的硬件和软件,如打印机只用 HP LaserJet,操作系统和浏览器只用 Windows XP 和 IE,无须对各种各样的硬件和软件进行兼容性测试。对于客户端软件,一般情况下,性能不是问题,而容错性、稳定性的测试则显得重要些。

对于企业级应用系统来说,存在着不同的应用模式,其系统的架构也不一样,可以分为以功能为中心、以数据库为中心和以业务逻辑(工作流)为中心等,在进行系统测试时,所设定的目标也有一定的区别。

(1) 以功能为中心的系统,强调模块化的低耦合性和高内聚性,这类系统的可扩充性、维护性要求很高。

(2) 以数据库为中心的系统,强调数据处理的性能、正确性和有效性,使数据具有良好的一致性和兼容性,同时,确保数据的安全性,包括数据的存储、访问控制、加密、备份和恢复等。

(3) 以应用逻辑(工作流)为中心的系统,强调灵活、流畅和时间性,系统的可配置性强,接口规范,如采用 XML 统一各工作流构件的输入和输出。

除此之外,还有其他一些因素的影响,如项目的周期性和依赖性等。如果项目是一次性的,对可扩充性、可移植性等要求低,而长期性的项目(如产品开发)对可扩充性、可移植性要求就很高。对软件即服务(Software as a Service, SaaS)的应用服务模式,对软件运行的服务质量(Quality of Service, QoS)也有很高的要求,需要支持 7×24 不间断的服务。对于这样的 Web 服务软件,非功能性测试的需求涉及性能、安全性、容错性、兼容性、可用性、可伸缩性等各个方面。

服务级别协议(Service Level Agreement, SLA)指定了最低性能要求,以及未能满足此要求时必须提供的客户支持级别和程度。与 QoS 要求一样,服务级别要求源自业务要求,对要求的测试条件及不符合要求的构成条件均有明确规定,并代表着对部署系统必须达到的整体系统特性的担保。服务级别协议被视为合同,所以必须明确规定服务级别要求。如表 10-1 所示,下面侧重性能、可用性、安全性和兼容性的测试需求讨论,而对其他非功能性属性就不进行过多的讨论,这并不意味着这些属性就没有测试需求,例如,可维护性(即系统维护的容易程度)的测试需求也是很多的,包括系统监视、日志文件、故障恢复、数据更新和备份等测试。

表 10-1 影响 QoS 要求的系统特性

系统质量	说 明
性能	指按用户负载条件对响应时间和吞吐量所做的度量
可用性	指对系统资源和服务可供最终用户使用的程度度量,通常以系统的正常运行时间来表示
可伸缩性	指随时间推移为部署系统增加容量(和用户)的能力。可伸缩性通常涉及向系统添加资源,但不应要求对部署体系结构进行更改
安全性	指对系统及其用户的完整性进行说明的复杂因素组合。安全性包括用户的验证和授权、数据的安全以及对已部署系统的安全访问
潜在容量	指在不增加资源的情况下,系统处理异常峰值负载的能力。潜在容量是可用性、性能和可伸缩性特性中的一个因素
可维护性	指对已部署系统进行维护的难易度,其中包括监视系统、修复出现的故障以及升级硬件和软件组件等任务

知识点：SaaS 可用性测试

可用性是指系统正常运行的能力或程度,在一定程度上也是系统可靠性的表现,可用性测试就基本等同于可靠性测试。可用性一般用正常向用户提供软件服务的时间占总时间的百分比来表示,即:

$$\text{可用性} = \frac{\text{正常运行时间}}{(\text{正常运行时间} + \text{非正常运行时间})} \times 100\%$$

系统非正常运行时间可能是因硬件、软件、网络故障或任何其他因素(如断电)所致,这些因素能让系统停止工作或连接中断不能被访问或性能急剧降低不能使用软件现有的服务等。

可用性指标一般要求达到 4 个或 5 个“9”,即 99.99% 或 99.999%。

(1) 如果可用性达到 99.99%, 对于一个全年不间断(7×24 的方式)运行的系统,意味着全年(525 600min)不能正常工作的时间只有 52min,不到 1h。

(2) 如果可用性达到 99.999%, 意味着全年不能正常工作的时间只有 5min。

所以一个系统的可用性达到 99.999%, 基本能满足用户的需求。当然,不同的应用系统,可用性要求是不一样的,非实时性的信息系统或一般网站要求都很低,可能在 99% 和

99.5%之间,而对一些军事系统,则要求很高,如美国防空雷达系统全年失效时间不超过两秒,可用性高达7个“9”之上,达99.999994%。

可用性测试就比较困难,不可能有足够的时间来进行测试,只能采用空间换时间的办法,例如,在高负载情况下进行为期一周或一个月的测试,以判断其可靠性。其次,就是对提高可靠性的措施进行测试,如故障转移的测试。

10.3 测试项目的估算与进度安排

在进行项目计划时,就要确定资源需求和安排进度,而这些工作依赖于对测试范围和工作量的估算。估算技术主要有经验估算法或专家评估法、对比分析法、工作任务分解方法和数学建模方法等。通过比较、调整使用不同技术导出的估算值,计划者更有可能得到精确的估算。软件项目估算永远不会是一门精确的科学,但将良好的历史数据与系统化的技术结合起来能够提高估算的精确度。

项目开始前的计划,对任务的测试需求有一个大体的认识,但深度不够,其估算缺乏精度,进度表可能只是一个时间上的框架,其中一定程度上是靠计划制定者的经验来把握的。随着时间的推移、测试的不断深入,对任务会有进一步的认识,对很多问题都不再停留在比较粗略的估算上,项目进度表会变得越来越详细、越准确。这也就是说,计划是一个过程,不仅仅是写成一个“测试计划书”文档。

10.3.1 测试工作量估算

测试的工作量是根据测试范围、测试任务和开发阶段来确定的。测试范围和测试任务是测试工作量估算的主要依据。如何确定测试范围已在10.2节做了充分的讨论,可以根据产品需求规格说明来决定。测试任务是由质量需求、测试目标来决定的,质量要求越高,越要进行更深、更充分的测试,回归测试的次数和频率也要加大,测试的工作量也要增加。处在不同的开发阶段,测试工作量的差异也很大。新产品第一个版本的开发过程,相对于以后的版本来说,测试的工作量要大一些。但也不是绝对的,例如,第1个版本的功能较少,在第2、3个版本中,增加了较多的新功能,虽然新加的功能没有第1个版本的功能多,但是在第2、3个版本的测试中,不仅要完成新功能的测试,还要完成第1个版本的功能回归测试,以确保原有的功能正常。

在一般情况下,一个项目要进行两三次回归测试。所以,假定一轮(Round)功能测试需要100个人日,则完成一个项目所有的功能测试肯定就不止100个人日,往往需要200~300多个人日。可以采用以下公式计算:

$$W = W_0 + W_0 \times R_1 + W_0 \times R_2 + W_0 \times R_3$$

(1) W 为总工作量, W_0 为一轮测试的工作量。

(2) R_1, R_2, R_3 为每轮的递减系数。受不同的代码质量、开发流程和测试周期等影响, R_1, R_2, R_3 的值是不同的。对于每一个公司来说,可以通过历史积累的数据获得经验值。

测试的工作量,还受自动化测试程度、编程质量、开发模式等多种因素影响。在这些影响的因素中,编程质量是主要的。编程质量越低,测试的重复次数可能就越多。回归测试的范围,在这三次中可能各不相同,这取决于测试结果,即测试缺陷的分布情况。如果缺陷多且分

布很广,所有的测试用例都要被再执行一遍。缺陷少且分布比较集中,可以选择部分或少数的测试用例作为回归测试所要执行的范围。

代码质量相对较低的情况下,假定 $R1$ 、 $R2$ 、 $R3$ 的值分别为 80%、60%、40%,若一轮功能测试的工作量是 100 个人日,则总的测试工作量为 280 个人日。如果代码质量高,一般只需要进行两轮回归测试, $R1$ 、 $R2$ 值也降为 60%、30%,则总的测试工作量为 190 个人日,工作量减少了 32%以上。

自动化程度越高,测试工作量就越低。由计算机运行的自动化脚本效率很高,能使实际测试执行的工作量大大降低。但是在很多情况下,测试自动化并不能大幅度降低工作量,因为测试脚本开发的工作量很大。也就是说,将总体的测试工作量前移了,从测试执行阶段移到测试脚本设计和开发的阶段,总体工作量没有明显降低。同时,由于自动化脚本可以重复使用,而且机器可以没日没夜地运行,回归测试就可以频繁进行,如每天可以执行一次,这样任何回归缺陷都可以即时发现,提高软件产品的质量。

工作量的估计是比较复杂的,针对不同的应用领域、程序设计技术、编程语言等,其估算方法是不同的。其估算可能要基于一些假定或定义。

(1) 效率假设。即测试队伍的工作效率。对于功能测试,这主要依赖于应用的复杂度、窗口的个数、每个窗口中的动作数目。对于容量测试,主要依赖于建立测试所需数据的工作量大小。

(2) 测试假设。目的是验证一个测试需求所需测试的动作数目,包括估计的每个测试用例所用的时间。

(3) 阶段假定。指所处测试周期不同阶段(测试设计、脚本开发、测试执行等)的划分,包括时间的长短。

(4) 复杂度假定。应用的复杂度指标和需求变化的影响程度决定了测试需求的维数。测试需求的维数越多,工作量就越大。

(5) 风险假定。一般考虑各种因素影响下所存在的风险,将这些风险带来的工作量设定为估算工作量之外的 10%~20%。

10.3.2 工作分解结构表方法

要做好测试工作量的估算,需要对测试任务进行细化,对每项测试任务进行分解,然后根据分解后的子任务进行估算。通常来说,分解的粒度越小,估算精度越高。可以再加上 10%~15% 的浮动幅度,来确定实际所需的测试工作量。比较专业的方法是工作分解结构表(Work Breakdown Structure, WBS),它按以下三个步骤来完成。

(1) 列出本项目需要完成的各项任务,如测试计划、需求和设计评审、测试设计、脚本开发、测试执行等。

(2) 对每个任务进一步细分,可进行多层次的细分,直到不能细分为止。如针对测试计划,首先可细分为:

- ① 确定测试目标。
- ② 确定测试范围。
- ③ 确定测试资源和进度。
- ④ 测试计划写作。
- ⑤ 测试计划评审。

(3) 列出需要完成的所有任务之后,根据任务的层次给任务进行编号,就形成了完整的工作分解结构表(如表 10-2 所示)。

表 10-2 测试工作分解结构表

1 测试计划	4 测试执行
1.1 确定测试目标	4.1 第 1 轮新功能测试
1.2 确定测试范围	4.2 性能测试
1.3 确定测试资源和进度	4.3 安全性测试
1.4 测试计划写作	4.4 安装测试
1.5 测试计划评审	4.5 第 2 轮回归新测试
2 需求 and 设计评审	4.6 升级和迁移测试
2.1 阅读文档以了解系统需求	4.7 最后一轮回归测试
2.2 需求规格说明书评审	5 测试环境建立和维护
2.3 编写/修改测试需求	5.1 软硬件购买
2.4 设计讨论	5.2 测试环境建立
2.5 设计文档评审	5.3 日常维护
3 测试设计和脚本开发	6 测试结果分析和报告
3.1 确定测试点	6.1 缺陷跟踪和分析
3.2 设计测试用例	6.2 性能测试结果分析
3.3 评审和修改测试用例	6.3 编写测试报告
3.4 设计测试脚本结构	7 测试管理工作
3.5 编写测试脚本基础函数	7.1 测试人员培训
3.6 录制测试脚本	7.2 项目会议
3.7 调试和修改测试脚本	7.3 日常管理
3.8 测试数据准备	

WBS 除了用表格的方式表达之外,还可以采用结构图的方式,那样会更直观、方便,如图 10-1 所示。

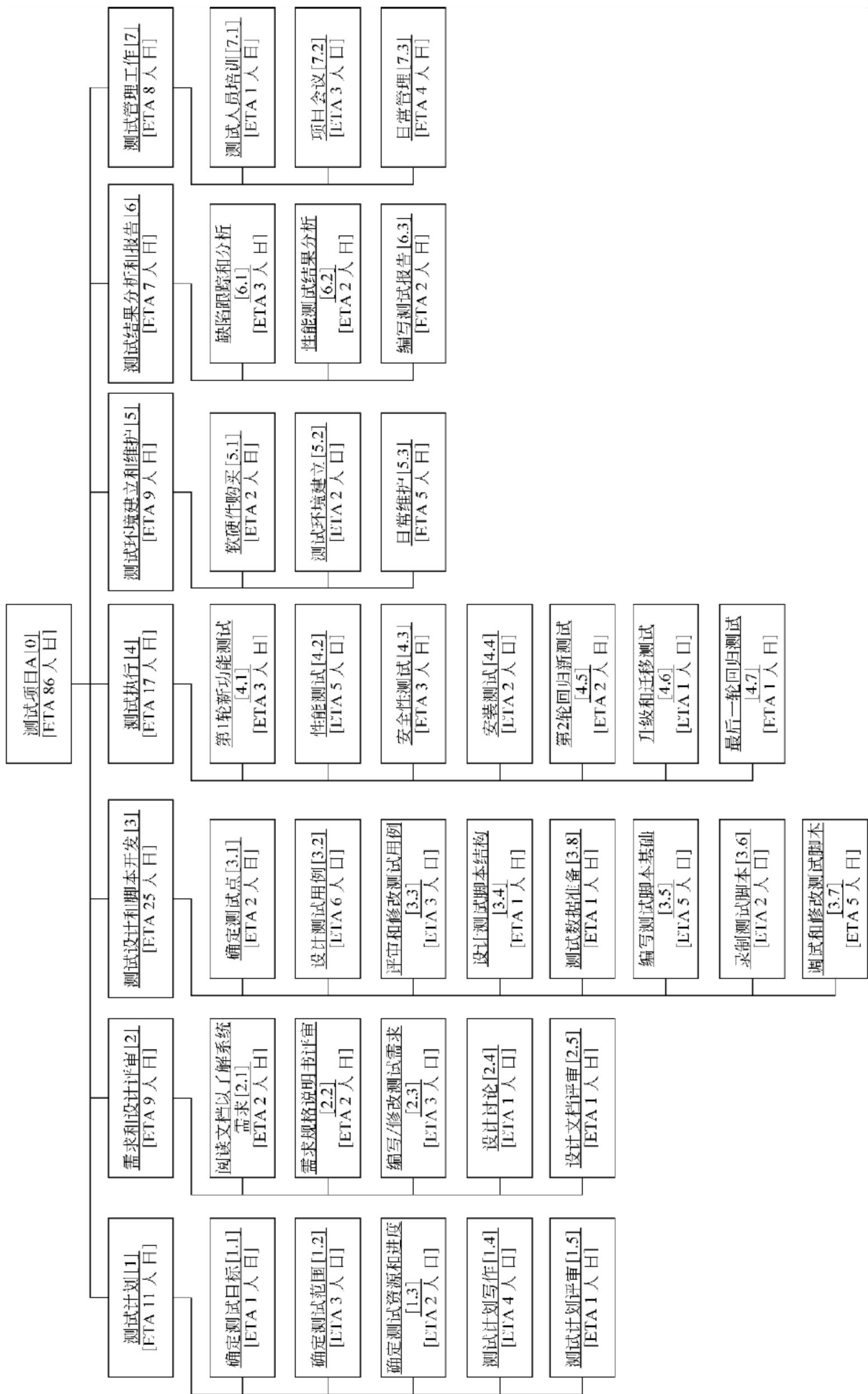
当 WBS 完成之后,就拥有了制定日程安排、资源分配和预算编制的基础信息,这样不仅可获得总体的测试工作量,还包括各个阶段或各个任务的工作量,有利于资源分配和日程安排。所以,WBS 方法不仅适合工作量的估算,还适合日程安排、资源分配等计划工作。

10.3.3 资源的安排

软件测试项目的资源管理是一项最基本的内容,项目的完成依赖于必要的资源,“巧妇难作无米之炊”,没有资源就无法去做事情。如果资源不够充分,项目能进行下去,但不能及时完成任务。资源管理的目的不仅要保证测试项目有足够的资源,同时,应能充分有效地利用现有资源,进行资源的优化组合,避免资源浪费。

测试项目的资源,主要分为人力资源、系统资源(硬件和软件资源)以及环境资源。每一类资源都由 4 个特征来说明:资源描述、可用性说明、需要该资源的时间以及该资源被使用的持续时间。后两个特征可以看成是时间窗口,对于一个特定的窗口而言,资源的可用性必须在开发的最初期就建立起来。

在完成了测试工作量估算之后就能够基本确定一个软件测试项目所需的人员数量,并写入测试计划中。但是,仅知道人员数量是不够的,因为软件测试项目所需的人员和要求在各个阶段是不同的。



(1) 在初期,也许只要测试组长介入进去,为测试项目提供总体方向、制定初步的测试计划,申请系统资源。

(2) 在测试前期,需要一些资深的测试人员,详细了解项目所涉及的业务和技术,分析和评估测试需求,设计测试用例、开发测试脚本。

(3) 在测试中期,主要是测试执行。如果测试自动化程度高,人力的投入没有明显的增加;如果测试自动化程度低,需要比较多的执行人员,他们也需要事先做好一定的准备。

(4) 在测试后期,资深的测试人员可以抽出部分时间去准备新的项目。

从经验看,人力资源的管理难度主要有以下三个方面。

(1) 资源需求的估计,依赖于工作量的估计和每个工程师的能力评估。

(2) 资源的应急处理,预留 10% 的资源作为人力储备(Buffer)。

(3) 资源的阶段间或多个项目间的平衡艺术。

10.3.4 测试里程碑和进度表

在软件测试项目的计划书中,都会制定一个明确的日程进度表。如何对项目进行阶段划分、如何控制进度、如何控制风险等有一系列方法,但最成熟的技术是里程碑管理。

里程碑是项目中完成阶段性工作的标志,即将一个过程性的任务用一个结论性的标志来描述任务结束的、明确的起止点,一系列的起止点就构成引导整个项目进展的里程碑(Milestone)。一个里程碑标志着上一个阶段结束、下一个阶段开始,也就是定义当前阶段完成的标准(Exit Criteria)和下个阶段启动的条件或前提(Entry Criteria)。里程碑还具有下列特征。

(1) 里程碑也是有层次的,在一个父里程碑内可以定义多个子里程碑;

(2) 不同类型的项目,可能设置不同类型或不同数量的里程碑;

(3) 不同规模项目的里程碑,其数量多少不一样,里程碑可以合并或分解。

在软件测试周期中,建议定义 6 个父里程碑、十几个子里程碑。

M1: 需求分析和设计的审查

M11: 市场/产品需求审查

M12: 产品规格说明书的审查

M13: 产品和技术知识传递

M14: 系统/程序设计的审查

M2: 测试计划 and 设计

M21: 测试计划的制定

M22: 测试计划的审查

M23: 测试用例的设计

M24: 测试用例的审查

M25: 测试工具的设计和选择

M26: 测试脚本的开发

M3: 代码(包括单元测试)完成

M4: 测试执行

M41: 集成测试完成

M42: 功能测试完成

M43: 系统测试完成

M44: 验收测试完成

M45: 安装测试完成

M5: 代码冻结

M6: 测试结束

M61: 为产品发布进行最后一轮测试

M62: 写测试和质量报告

对每个子里程碑,如果需要更加严格的控制,还可以定义更细的里程碑,见表 10-3。

表 10-3 软件测试进度表例子

任务	天	任务	天	任务	天	任务	天
M21: 测试计划制定	11	M23: 测试设计	12	开发测试过程	5	验证测试结果	2
确定项目	1	测试用例的设计	7	测试和调试测试过程	2	调查突发结果	1
定义测试策略	2	测试用例的审查	2	修改测试过程	2	生成缺陷日记	1
分析测试需求	3	测试工具的选择	1	建立外部数据集	1	M62: 测试评估	3
估算测试工作量	1	测试环境的设计	2	重新测试并调试测试过程	2	评估测试需求的覆盖率	1
确定测试资源	1	M26: 测试开发	15	M42: 功能测试	9	评估缺陷	0.5
建立测试结构组织	1	建立测试开发环境	1	设置测试系统	1	决定是否达到测试完成的标准	0.5
生成测试计划文档	2	录制和回放原型过程	2	执行测试	4	测试报告	1

10.4 测试风险和测试策略

测试总是存在着风险,软件测试项目的风险管理尤为重要,应预先重视风险的评估,并对要出现的风险有所防范。在风险管理中,首先要将风险识别出来,特别是确定哪些是可避免的风险,哪些是不可避免的风险,对可避免的风险要尽量采取措施去避免,所以风险识别是第一步,也是很重要的一步。风险识别的有效方法是建立风险项目检查表,按风险内容进行分项检查,逐项检查。然后,对识别出来的风险进行分析,主要从下列 4 个方面进行分析。

(1) 发生的可能性(风险概率)分析,建立一个尺度表示风险可能性(如极罕见、罕见、普通、可能、极可能);

(2) 分析和描述发生的结果或风险带来的后果,即估计风险发生后对产品和测试结果的影响、造成的损失等;

(3) 确定风险评估的正确性,要对每个风险的表现、范围、时间做出尽量准确的判断;

(4) 根据损失(影响)和风险概率的乘积,排定风险的优先队列。

评估方法可以采用情景分析和专家决策方法、损失期望值法、风险评审技术、模拟仿真法和 FMEA(Failure Mode and Effects Analysis,失效模型和效果分析)法等。

10.4.1 测试风险管理计划

为了避免、转移或降低风险,事先要做好风险管理计划,包括单个风险的处理和所有风险综合处理的管理计划。风险的控制是建立在上述风险评估的结果上,对风险的处理还要制定一些应急的、有效的处理方案,不同类型的风险,对策也是不同的。

(1) 采取措施避免那些可以避免的风险,如可以通过事先列出要检查的所有条目,在测试环境设置好后,由其他人员按已列出条目逐条检查,避免环境配置错误。

(2) 风险转移,有些风险可能带来的后果非常严重,能否通过一些方法,将它转化为其他一些不会引起严重后果的低风险。如产品发布前夕发现,由于开发某个次要的新功能,给原有的功能带来一个严重 Bug,这时要修正这个 Bug 所带来的风险就很大,采取的对策就是关闭(不激活)那个新功能,转移修正 Bug 的风险。

(3) 有些风险不可避免,就设法降低风险,如“程序中未发现的缺陷”这种风险总是存在,就要通过提高测试用例的覆盖率(如达到 99.9%)来降低这种风险。

风险管理的完整内容和对策,如图 10-2 所示。控制风险还有一些其他策略,如:

(1) 在做计划时,估算资源、时间、预算时,要留有余地,如增加 10%~15% 的额度;而且把一些环节或边界上的有变化、难以控制的因素列入风险管理计划中。

(2) 对每个关键性技术人员培养后备人员,做好人员流动的准备,采取一些措施确保某些关键人员一旦离开公司,项目不会受到严重影响,仍可以继续下去。

(3) 制定文档标准,并建立一种机制,保证文档及时产生。

(4) 对所有工作实施互相审查机制,及时发现问题。

(5) 对所有过程进行日常跟踪,及时发现风险出现的征兆,避免风险。

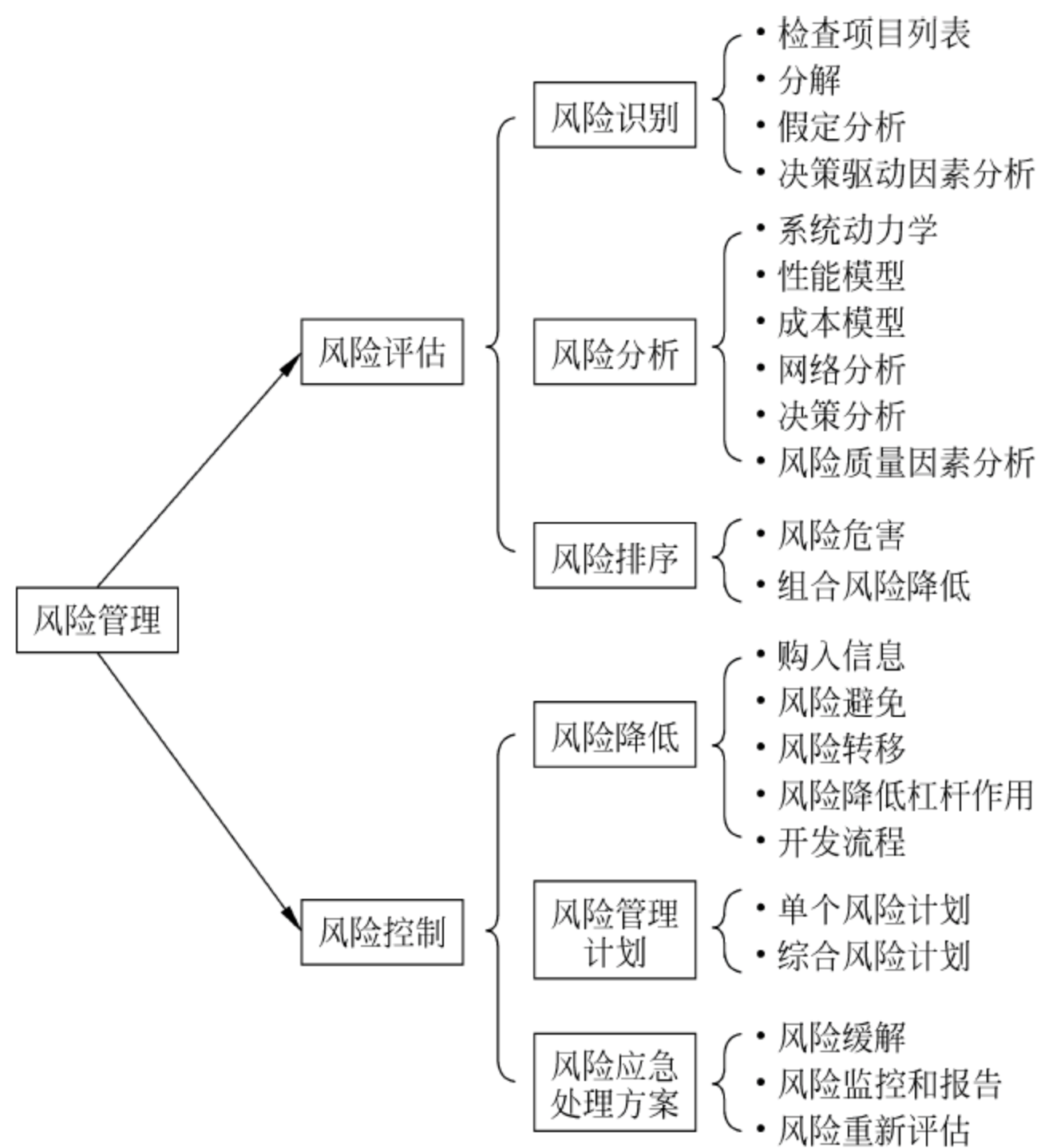


图 10-2 风险管理的内容和对策

10.4.2 测试策略的确定

软件测试通常指实际运行被测程序,输入相应的测试用例,判定执行结果是否符合要求,从而检验程序的正确性、可靠性和有效性。软件测试可采用的方法和技术是多种多样的,但通

常情况下不论采用什么方法和技术,其测试都是不彻底的,也是不完全的,因为任何一次完全测试或者穷举测试(即让被测程序在一切可能的操作情况下,包括正确的操作,也包括错误的操作情况下,全部执行一遍)的工作量太大,在实践上行不通。因此,任何实际测试,都不能够保证被测试程序中不存在遗漏的缺陷。

为了最大程度地减少这种遗漏的错误,同时也为了最大限度地发现已经存在的错误,在测试实施之前,必须确定采用合适的测试策略和测试方法,并以此为依据制定详细的测试用例。而一个好的测试策略和测试方法,必将给软件测试带来事半功倍的效果,它可以充分利用有限的人力和物力资源,高效率、高质量地完成测试。

由此可知,测试策略通常是描述测试项目的目标和所采用的测试方法,确定在不同的测试阶段测试范围、测试任务的优先级,以及所采用的测试技术和工具,以获得最有效的测试和可能达到的质量水平。在制定测试策略前,要认真分析测试策略影响因素,例如:

(1) 要使用的测试技术和工具,如自动化测试比例要达到 60%,手工测试是 40%。

(2) 测试完成标准。每个具体软件都有其特殊性,测试完成的标准会有差异,测试完成的标准对策略确定有着重要的影响,标准高,测试工作量会增大,在资源有限的情况下,应借助有效的方法获得可接受的产品质量水平。例如,军用系统对软件的可靠性、安全性要求非常高,而小型商场的收费系统,则强调数据的准确性、管理的灵活性和易用性。

(3) 影响资源分配的特殊考虑,例如,有些测试必须在周末进行,有些测试必须通过远程环境执行,有些测试需考虑与外部接口或硬件接口集成后进行。在测试资源分配时,考虑测试需求,有针对性地采取不同的测试方法和时间,会节省一定的测试资源。

依据软件本身的性质、规模及应用的场合不同,将选择不同的测试方案,以最少的软件、硬件及人力资源的投入得到最佳的测试效果,这就是测试策略目标所在。通过以上分析,可以对测试策略的确定过程归纳为输入、输出和过程。

1. 输入

- (1) 要求的硬件和软件组件的详细说明,包括测试工具(测试环境,测试工具数据);
- (2) 针对测试和进度约束(人员,进度表)而需要的资源的角色和职责说明;
- (3) 测试方法(标准);
- (4) 应用程序的功能性和技术性需求(需求,变更请求,技术性和功能性设计文档);
- (5) 系统不能够提供的需求(系统局限)。

2. 输出

- (1) 已批准和签署的测试策略文档,测试计划,测试用例;
- (2) 需要解决方案的测试项目(通常要求客户项目的管理层协调)。

3. 过程

测试策略是关于如何测试系统的正式描述,要求开发针对所有测试级别的测试策略。测试小组分析需求,编写测试策略并且和项目小组一起复审计划。测试计划应该包括测试用例和条件、测试环境、与任务相关的测试、通过/失败的准则和测试风险评估。测试进度表将识别所有要求有成功的测试成果的任务,活动的进度和资源要求。

那么,究竟如何才能确定一个好的测试策略和测试方法呢?常用的策略有基于测试技术的测试策略和基于测试方案的综合测试策略。

- (1) 基于测试技术的测试策略。著名软件测试专家 Myers 指出了使用各种测试方法的综

合策略:

① 在任何情况下都必须使用边界值分析方法。经验表明,用这种方法设计出测试用例发现程序错误的能力最强。

② 必要时用等价类划分方法补充一些测试用例。

③ 用错误推测法再追加一些测试用例。

④ 对照程序逻辑,检查已设计出的测试用例的逻辑覆盖程度。如果没有达到要求的覆盖标准,应当再补充足够的测试用例。

⑤ 如果程序的功能说明中含有输入条件的组合情况,则一开始就可选用因果图法。

(2) 基于测试方案的综合测试策略。一方面,可根据程序的重要性和一旦发生故障将造成的损失,来确定它的测试等级和测试重点;另一方面,通过认真研究分析,使用尽可能少的测试用例,发现尽可能多的程序错误,因为一次完整的软件测试过后,如果程序中遗漏的错误过多并且很严重,则表明本次测试是失败的,是不足的;而测试不足意味着让用户承担隐藏错误带来的危险。但是,如果过度测试,则又会造成资源浪费。我们需要在这两点上权衡,找到一个最佳平衡点。

10.5 测试计划的内容与编制

软件项目计划的目标是提供一个框架,不断收集信息,对不确定性进行分析,将不确定性的内容慢慢转化为确定性的内容,该过程最终使得管理者能够对资源、成本及进度进行合理的估算。这些估算还是在项目早期做出的,并受到时间的限制,所以计划能接受一定的风险和不确定性,今后还会随着项目的进展而不断更新。

10.5.1 测试计划内容

软件测试计划是指导测试过程的纲领性文件,描述测试活动的范围、方法、策略、资源、任务安排和进度等,并确定测试项、哪些功能特性将被测试、哪些功能特性将无须测试,识别测试过程中的风险。借助软件测试计划,确保测试实施过程顺畅,能有效地跟踪和控制测试过程,并能容易应对可能发生的各种变更。

在制定测试计划时,由于不同软件公司的背景不同,测试计划内容会有差异,但一些基本内容是相同的。例如,IEEE 829—1998 软件测试文档编制标准中规定软件测试计划应包含如下 16 项内容。

- (1) 测试计划标识符(文档编号);
- (2) 项目总体情况简介;
- (3) 测试项(Test Item);
- (4) 需要测试的功能;
- (5) 方法(策略);
- (6) 不需要测试的功能;
- (7) 测试项通过/失败的标准;
- (8) 测试中断和恢复的规定;
- (9) 测试完成所提交的材料;
- (10) 测试任务;

- (11) 测试环境要求;
- (12) 测试人员职责;
- (13) 人员安排与培训需求;
- (14) 进度表;
- (15) 潜在的问题和风险;
- (16) 审批。

在测试计划中,还要考虑休假和法定假日带来的影响,以及做好项目相关技术和业务的培训。软件测试计划内容主要集中在测试目标和需求说明、测试工作量估算、测试策略、测试资源配置、进度表、测试风险等。

(1) 目标和范围:包括质量目标、产品特性、各阶段的测试对象、目标、范围和限制。

(2) 项目估算:根据历史数据和采用恰当的评估技术,如项目工作分解结构方法,对测试工作量、所需资源(人力、时间、软硬件环境)做出合理的估算。

(3) 风险计划:测试可能存在的风险分析、识别,以及风险的回避、监控和管理。

(4) 进度表:根据项目估算结果和人力资源现状,以软件测试的常规周期作为参考,采用关键路径法等,完成进度的安排,采用时限图、甘特图等方法来描述资源和时间的关系,例如,什么时候测试哪一个模块、什么时候要完成某项测试任务等。

(5) 项目资源:人员、硬件和软件等资源的组织和分配,人力资源是重点,它和日程安排联系密切。

(6) 跟踪和控制机制:质量保证和控制,变化管理和控制等。例如,明确如何提交一个问题报告、如何去界定一个问题的性质或严重程度、多少时间内做出响应等。

对于大型软件项目,可能需要一系列测试计划书,例如,按集成测试、系统测试、验收测试等阶段去组织,为每一个阶段制定一个计划书,也可以为每个测试项(安全性测试、性能测试、可靠性测试等)制定特别的计划书,甚至可以制定测试范围/风险分析报告、测试标准工作计划、资源和培训计划、风险管理计划、测试实施计划、质量保证计划等。

10.5.2 测试项目的计划过程

测试项目的计划不可能一气呵成,而是要经过计划初期、起草、讨论、审查等不同阶段,才能将测试计划制定好。测试计划的编写是一项系统工作,编写者随着对项目逐步了解,不断细化和完善测试计划。一般来说,在测试需求分析前制作总体测试计划书,在测试需求分析后制作详细测试计划书。

(1) 计划初期是收集整体项目计划、需求分析、功能设计、系统原型、用户用例(Use Case)等文档或信息,理解用户的真正需求,了解技术难点和弱点,并与其他项目相关人员进行充分交流,在需求和设计上达到一致的理解。

(2) 测试计划最关键的一步就是确定测试需求和测试范围,将软件系统逐步分解,将软件分解为较小而且相对独立的功能模块。这样,针对各个单元就比较容易写成测试需求,测试需求是测试用例设计的基础,并用来衡量测试覆盖率的重要指标。

(3) 计划起草。根据计划初期所掌握的各种项目信息,确定测试策略,选择合适、有效的测试方法,完成测试计划的框架。

(4) 内部审查。在提供给其他部门讨论之前,先在测试小组/部门内部进行审查,测试团队的其他人员帮助发现问题,并在测试团队内部达成一致。

(5) 计划讨论和修改。召开有需求分析、设计、开发人员参加的计划讨论会议,测试组长对测试计划设计的思想、策略做较详细的介绍,并听取大家对测试计划中各个部分的意见,进行讨论交流。

(6) 测试计划的多方审查。计划的审查是必不可少的,尽管测试团队努力制定一个全面的、有效的测试计划,但还会受到测试团队本身的局限性,测试计划可能不够完整、准确。此外,就像开发者很难测试自己的代码那样,测试工程师也很难评估自己的测试计划。项目中的每个团队都应派人参与测试计划的审查,每个审查者都可能根据其经验及专长发现测试计划中的问题,提出良好的建议。

(7) 测试计划的定稿和批准。在计划讨论、审查的基础上,综合各方面的意见,就可以完成测试计划书,然后上报给测试经理或更高层的经理,得到批准,方可执行。

(8) 测试计划的跟踪。测试计划书完成之后,不要束之高阁,而是要跟踪其执行,随时将测试执行状态和测试计划要求进行比对。如果是执行问题,就需要纠正执行;如果是计划跟不上需求和设计的变化,就要对计划做相应的调整。一个好的测试计划,和实际执行的偏差不应该太大,理想情况下,两者保持一致。

在测试计划的每个阶段上,都要清楚该阶段要达到的目标、负责人是谁、哪些人要参与(提供信息、参加评审等)、工作的重点是什么、最终需要提供哪些资料。例如,以计划起草阶段为例:

(1) 目标:重点描述如何使测试建立在客观的基础上,定义测试项及其基本方法、策略,粗略估算测试需要的时间和人力资源周期、最终递交测试报告的时间等。

(2) 工作重点:绘制一个相对完整的功能结构图,并描述功能特性测试会覆盖哪些功能特性,如果没有覆盖全部的功能特性,那么会带来哪些测试风险或多大的测试风险;如何验证系统设计、验证设计需要多长时间,在此之前,是否需要测试人员进行相关培训;根据系统平台选型,如何搭建测试平台。

(3) 需要的资料:项目的整体计划书初稿、产品需求文档初稿、用例(Use Case)和其他项目文档等。

(4) 成果:测试计划书初稿、系统功能结构图等。

① 负责人:测试组长。

② 参与人:市场部门、产品经理、项目经理、开发组长和测试组其他人员。

③ 变更:说明有可能导致测试计划变更的事件,包括项目整体计划的变更、增加新的功能特性、测试工具的改进、测试环境的改变等。

测试计划不仅是软件产品当前版本而且还是下一个版本的测试设计的主要信息来源,在进行新版本测试时,可以在原有的软件测试计划书上进行修改来完成计划的制定,会节约比较多的时间。

10.5.3 制定有效的测试计划

在计划书中,有些内容是介绍测试项目的背景、所采用的技术方法等,这些内容仅作为参考,但有些内容则可以看作是测试组所做出的承诺,必须要实施或达到的目标,如要完成的测试任务、测试组构成和资源安排、测试项目的里程碑、面向解决方案的交付内容、项目标准、质量标准、相关分析报告等。

要做好测试计划,测试设计人员要仔细阅读有关资料,包括用户需求规格说明书、设计文

档、使用说明书等,全面熟悉系统,并对软件测试方法和项目管理技术有着深刻的理解,并建议注意以下方面。

- (1) 确定测试项目的任务,清楚测试范围和测试目标,如提交什么样的测试结果。
- (2) 测试计划尽量识别出各种测试风险,并制定出相应的对策。
- (3) 让所有合适的相关人员参与测试项目的计划制定,特别是在测试计划早期。
- (4) 对测试的各阶段所需要的时间、人力及其他资源进行预估,尽量做到客观、准确、留有余地。
- (5) 制定测试项目的输入、输出和质量标准,并和有关方面达成一致。
- (6) 建立变化处理的流程规则,识别出在整个测试阶段中哪些是内在的、不可避免的变化因素,如何进行控制。
- (7) 不要忽视技术上的问题,例如,系统架构的设计对系统的性能测试、故障转移测试等有较大影响。在测试计划过程中,要和系统设计人员充分沟通。
- (8) 要对测试的公正性、遵照的标准做一个说明,证明测试是客观的,整体上,软件功能要满足需求,实现正确,和用户文档的描述保持一致。
- (9) 测试计划应简洁、易读并有所侧重,重点内容要详细描述,避免测试计划的“大而全”、重点不突出和缺乏层次。例如,具体的测试技术指标可以用单独的测试详细规格文档来说明,通用测试流程也应该由单独文档来描述,而测试用例就更不要放在测试计划中。

小结

本章主要讨论测试需求和如何创建有效的测试计划。测试需求包括功能测试需求和非功能性测试需求,而非功能性测试需求包括性能、安全性、可靠性、兼容性、易维护性和可移植性等测试需求。对于非功能性测试需求,既要独立考虑它们各自的特点和各自的测试需求,也要考虑它们之间的关系和相互影响,例如安全性和可靠性密切相关,越安全越可靠,越可靠越安全。而安全性会增加许多保护措施,往往会降低性能。在整个系统测试需求分析时,不仅要考虑来自整体系统的测试需求,还要考虑系统数据、外部接口等测试需求。而在测试计划过程中,主要做好下列各项工作。

- (1) 确定软件功能性、非功能性的测试需求,以及各个阶段的测试任务。
- (2) 进行测试范围分析,从而对测试工作量进行估算。工作量估算方法主要介绍了工作分解结构表方法,并给出了实例。
- (3) 测试资源需求、团队组建,包括培训。
- (4) 测试里程碑和进度的安排。
- (5) 对测试风险进行分析。
- (6) 制定有效的测试策略。

最后完整地生成测试计划书,进行计划书的评审、跟踪和及时修改,测试计划是一个过程,不仅是“测试计划书”这样一个文档,测试计划会随着情况的变化不断进行调整,用以优化资源和进度安排,降低风险,提高测试效率。

思考题

1. 当被指定为某软件测试项目的组长,为某个测试项目制定测试计划时,该做哪些准备?
2. 就某个具体项目,如何有效开展测试需求分析? 并进一步识别测试需求和产品需求的不同点。
3. 测试工作量估算是比较困难的,除了 WBS 方法之外,还有什么有效方法?
4. 结合某个具体项目,完成一个完整的测试计划书的编制。